ShipShape Audit — Findings Summary

Phase 1 (Diagnosis) · branch `shipshape/audit` · 7/7 categories baselined · companion to `AUDIT_REPORT.md` · each table uses the PRD's prescribed metrics; each category includes **improvement options** (audit opportunities — no code changed)

> **Condition of record (all baselines) — SNAPSHOT-PINNED:** documents=627, issues=328, sprints=35, users=31, associations=625. Frozen pg_dump at `scripts/audit/snapshot/ship_dev.condition.dump`; restore with `bash scripts/audit/db-restore.sh` (round-trip validated). Node v20.20.2 · PostgreSQL 16 (Docker) · macOS aarch64. Every number is produced by a committed re-runnable script in `scripts/audit/` → raw in `docs/audit/raw/`. **Data-safety:** `pnpm --filter @ship/api test` truncates `ship_dev` — restore from the snapshot after.

1 — Type Safety measured

TypeScript Compiler API AST walk (no linter in repo; `strict` ON). Scope A = non-test src. → `cat1-type-safety.mjs`

Total any types	**94** (api 65 · web 29 · shared 0)
Total type assertions (as)	**433** (api 135 · web 298 · shared 0)
Total non-null assertions (!)	**325** (api 292 · web 33 · shared 0)
Total `@ts-ignore` / `@ts-expect-error`	**0** (1 in tests only)
Strict mode enabled?	**Yes** (strict + noUncheckedIndexedAccess + noImplicitReturns + noFallthroughCasesInSwitch; web tsconfig omits last 3)
Strict mode error count (if disabled)	N/A (strict enabled)
Top 5 violation-dense files	weeks.ts (84) · projects.ts (51) · issues.ts (48) · UnifiedDocumentPage.tsx (37) · seed.ts (35)

Aggregate = **852** violations; PRD Improvement Target "eliminate 25%" ⇒ ≤ **639** (–213).

1. **High** — non-null `!` (325, api 292) is the dominant risk class.
2. **High** — `as` (433, web 298): downstream loss of discriminated-union narrowing.
3. **Medium** — explicit `any` (94) in api routes; **no linter** guards new violations.

> ▸ **Improve** — PRD target: eliminate **25%** (852 → ≤639), real fixes only.
>
> - **A L-risk** remove non-null `!` in api hot files (weeks 48 · issues 37 · seed 35 · team 28 = ~148) via real guards/early-returns.
> - **B M** replace web `as` casts (UnifiedDocumentPage/Editor/PropertiesPanel ~88) by narrowing on the `document_type` union.
> - **C L** add `@typescript-eslint` (no-non-null-assertion / no-explicit-any) as errors → durable guard.
> - **D M** type the `y-protocols.d.ts` shim + api-route `any` (~40) with real types.
>
> **Recommended: A + C** — clears the count fast + stops regressions. Verify `cat1-type-safety.mjs` before/after.

2 — Bundle Size measured

Production `vite build --sourcemap` + sourcemap byte attribution. → `cat2-bundle.mjs`

Total production bundle size	**2,275 KB raw / 695.5 KB gzip** (JS 2,210 / CSS 65)
Largest chunk (name + size)	`index-C2vAyoQ1.js` — **2,025 KB raw / 575.7 KB gzip**
Number of chunks	261 JS + 1 CSS — **but 91.6% of all JS is in that one chunk**
Top 3 largest dependencies	emoji-picker-react (~398 KB, 7.8%) · highlight.js (~376 KB, 7.4%) · react-router (~347 KB, 6.8%)
Unused dependencies identified	`@tanstack/query-sync-storage-persister` (candidate — verify)

Code splitting (PRD "How to Measure"): minimal — 3 `React.lazy`; tab chunks 1–16 KB; everything else monolithic. PRD Improvement Target = −15% total *or* −20% initial via splitting.

1. **High** — monolithic entry chunk (576 KB gzip, 91.6% of JS); no route/vendor split.
2. **High** — editor-only deps (highlight.js, emoji-picker) ship on first load; lazy-load = large low-risk win.

▸ **Improve** — PRD target: −15% total *or* −20% initial via splitting (no functionality removed).

- **A** **L** lazy `import()` editor-only deps (emoji-picker ~398 KB + highlight.js ~376 KB) — load only when editor mounts (~−25–35% initial).
- **B** **L** highlight.js: register only used languages, not the full bundle (~−300 KB raw).
- **C** **M** route-level `React.lazy`+Suspense for top-level pages.
- **D** **L** Vite `manualChunks` vendor split (react / tiptap+prosemirror / yjs) for caching.

Recommended: A + B — clears −20% initial at lowest risk. Verify `cat2-bundle.mjs` before/after.

3 — API Response Time measured

Dependency-free concurrent-load harness, API :3000 direct, warmup=10/budget=120/cell @ concurrency 10·25·50 (headline @25). → `cat3-api.mjs`

Endpoint (key flow) @conc 25	P50	P95	P99
`GET /api/documents?document_type=wiki` (main page, ~300 KB)	185ms	**243ms**	254ms
`GET /api/issues` (list issues, ~280 KB, 328 rows)	104ms	**123ms**	143ms
`GET /api/weeks` (sprint board)	24ms	29ms	30ms
`GET /api/search/mentions?q=load` (search)	17ms	21ms	22ms
`GET /api/documents/:id` (view document)	33ms	44ms	48ms

Concurrency tested 10/25/50, 0 errors all cells. P95 scales ~linearly on the slow two: main_page 112→243→**420ms**, list_issues 69→123→**229ms** (snapshot-pinned, matches `cat3-before.txt`).

1. **High** — two unbounded list endpoints dominate & degrade ~linearly; SQL <1.5 ms ⇒ cost = serializing 300/280 KB JSON on the shared event loop.
2. **Medium** — global rate limiter **100 req/min prod**; per-request session write on every endpoint.

▸ **Improve** — PRD target: **−20% P95** on ≥2 endpoints, identical conditions, root cause documented.

- **A** **M** drop heavy `content`/`yjs_state` blobs from the *list* responses of `/api/documents` & `/api/issues` (root cause = JSON serialization) → large P95 drop on both.
- **B** **M** add `LIMIT` + cursor pagination to the same two endpoints.
- **C** **L** enable `compression` middleware + cache headers on these GETs.
- **D** **H** stream/offload serialization off the shared event loop.

Recommended: A + C — A hits ≥2 endpoints at once; C is free margin. Verify `cat3-api.mjs` before/after.

4 — Database Query Efficiency measured

Postgres full statement logging; 5 marker-bracketed flows, API-pool PIDs only. → `cat4-db.mjs`

User Flow	Total Queries	Slowest (ms)	N+1?
Load main page `/api/documents?type=wiki`	4	1.61	No
View a document `/api/documents/:id`	4	0.369	No
List issues `/api/issues`	5	1.158	No
Load sprint board `/api/weeks`	5	0.382	No
Search content `/api/search/mentions`	5	0.470	No

EXPLAIN: main_page exec 0.144 ms / planning 0.561 ms; purpose-built partial index unused at this volume.

1. **High** — universal auth tax: every flow runs `SELECT sessions` + `UPDATE sessions.last_activity` — 2 of every 4–5 queries are auth overhead.
2. **Medium** — no pagination/`LIMIT`; grows linearly with workspace size. **Low** — well-indexed, no N+1.

▸ **Improve** — PRD target: **–20% query count** on ≥1 flow *or* –50% on slowest query (before/after EXPLAIN).

- **A L** throttle the per-request `UPDATE sessions.last_activity` (write only if >60 s stale) → view_document 4→3 = **–25%**, every flow loses 1 query.
- **B H** short-TTL session cache to also drop the per-request `SELECT sessions` (–50%; revocation-lag risk).
- **C M** collapse SELECT+UPDATE into one `UPDATE … RETURNING`.
- **D M** pagination/`LIMIT` (shared w/ Cat-3 B) to bound rows scanned.

Recommended: A — lowest risk, exactly clears –20%, compounds Cat-3. Verify `cat4-db.mjs` before/after.

5 — Test Coverage & Quality measured

Unit run 3× for flakiness; E2E static catalog; coverage attempted. → `docs/audit/raw/cat5-before.txt`

Total tests	**451 unit (api)** + **151 unit (web)** + **~882 E2E** = ~1,484 total (vs README "73+")
Pass / Fail / Flaky	**api: 451 / 0 / 0** on fresh seed; **8 fail on the locked snapshot** (test-isolation defect). **web: 138 / 13 / —** (TipTap/ProseMirror, hidden by default `pnpm test`). E2E: not obtainable.
Suite runtime	unit: ~15–16 s (16/15/16). E2E: not obtainable
Critical flows with zero coverage	document CRUD via HTTP, auth, real-time Yjs collaboration; 16 web unit files never run by `pnpm test`
Code coverage % (if measured)	**api Lines 40.52%** · Stmts 40.34% · Funcs 40.9% · Branches 33.44%. **web Lines 28.53%** · Stmts 27.63% · Funcs 25.6% · Branches 19.38%. Configured per PRD instruction (#3b: added `@vitest/coverage-v8@4.0.17`; absent as-shipped — finding kept).

1. **High** — mandated `/e2e-test-runner` skill **does not exist**; 882-test suite unrunnable by sanctioned method.
2. **High** — coverage unmeasurable as-shipped. **Positive** — unit layer stable & fast.

▸ **Improve** — PRD target: meaningful tests for **3** untested critical paths *or* fix 3 flaky w/ root cause (each test commented w/ risk it mitigates).

- **A M** add api integration tests for 3 untested paths: document CRUD over HTTP · auth/session lifecycle & timeout · Yjs collab persistence (locks in Cat-6 fix).
- **B L** isolated unit-test DB (separate DATABASE_URL) so tests stop truncating `ship_dev` — unblocks safe repeated runs.
- **C L** install `@vitest/coverage-v8` + web coverage config → baseline measurable.
- **D H** implement the missing `/e2e-test-runner` skill → 882-test E2E runnable safely.

Recommended: B then A — B so tests don't destroy seed data, then A meets the bar. Verify `pnpm --filter @ship/api test`.

6 — Runtime Errors & Edge Cases measured

Reproducible Playwright harness: console · malformed input · RT1 collab (validated control) · slow-3G · PG log. → `cat6-runtime.mjs`

Console errors during normal usage	**0** across all 6 pages (0 console.error / pageerror / failed req / ≥500)
Unhandled promise rejections (server)	Not directly observable (**no centralized API logging**); proxy: 1 Postgres ERROR (the bad-UUID 500); 0 browser 5xx in normal nav
Network disconnect recovery	**Pass** — online control PERSISTED; transient disconnect (tab open) PERSISTED; Yjs IndexedDB offline store present
Missing error boundaries	None triggered in normal use (0 pageerrors); fault-injection not exercised this pass
Silent failures identified	`text/plain` body → **201** creates a default doc; + RT1/RT3 server-side swallowed persist

Malformed-input matrix: bad JSON → 400 HTML stack page · missing CSRF → 403 HTML stack · oversized title → 400 clean JSON ✅ · wrong content-type → 201 ⚠️ · bad UUID path → 500 + raw PG error ⚠️

1. **High** — unvalidated path params → raw DB error as 500; bad-JSON/missing-CSRF leak HTML stack traces (inconsistent contract).
2. **Medium** — no Content-Type guard (silent doc creation). **RT1 honestly downgraded**: client offline path robust; residual = server-side swallowed persist.

▸ **Improve** — PRD target: fix **3** gaps; **≥1 must be real user-facing data-loss/confusion** (repro + before/after + recording).

- **A M (data-loss)** surface the swallowed Yjs server-persist failure — stop the silent `catch`, signal client (toast+retry) & log; users currently believe failed saves succeeded.
- **B L** validate path params (UUID/zod) → **400/404** instead of raw PG error as 500.
- **C M** global JSON error envelope + prod handler → no HTML stack traces, consistent `{error}`.
- **D L** enforce Content-Type on writes → stop silent junk-doc creation (201).

Recommended: A + B + C — exactly 3; A is the mandatory data-loss fix. Verify `cat6-runtime.mjs` + repro.

7 — Accessibility measured

THREE harnesses, same 6 pages: Lighthouse 13 a11y (3-run median, JSON+HTML in `reports/a11y/before/`) + axe-core WCAG 2.1 A+AA + keyboard/contrast/landmark (validated by an unauthenticated login control) + **SR-tree proxy** via CDP `Accessibility.getFullAXTree` (Q3-C: same tree VoiceOver/NVDA read from). → `cat7-lighthouse.mjs` + `cat7-a11y.mjs` + `cat7-sr-tree.mjs`. Lighthouse/axe are automated (~30–40% of WCAG) and do NOT test keyboard traversal or SR navigation flow — high scores ≠ compliant.

Lighthouse accessibility score (per page)	**median of 3 runs: login 98 · main_docs 91 · view_document 91 · issues 100 · my_week 96 · team_dir 100** (lowest = 91, main_docs & view_document). Full JSON+HTML in `reports/a11y/before/`. Plus axe violations/passes: login 0/23 · main_docs 2/22 · view_document 2/22 · issues 0/21 · my_week 1/20 · team_dir 0/20. *Automated scores miss keyboard — a 100 is not a victory.*
Total Critical/Serious violations	**Critical = 2, Serious = 17** node instances (C+S = 19); 3 rules: aria-required-children, listitem, color-contrast
Keyboard navigation completeness	Auto-opening "Action Items" overdue-standup modal (visible Radix dialog — screenshot `reports/a11y/before/cat7-docs-modal.png`) confines focus to 3 buttons (60 Tabs → 3 vs 4/4 login). **`Escape`/`Got it` dismisses it → WCAG 2.1.2 PASSES.** Appears for any user with a pending/overdue item (mid-week, most). Defect = the disruptive auto-open, not a broken keyboard.
Color contrast failures	**15** nodes, all on `/my-week` (low-opacity muted text) — WCAG 1.4.3 AA
Missing ARIA labels or roles	aria-required-children (critical) + listitem (serious) on main_docs & view_document; login lacks main/nav landmarks; `lang=en` ✅ . **Editor (view_document):** contenteditable body has no role/aria-label — nameless edit region for SR.
SR a11y-tree — modal OPEN vs DISMISSED (Q3-C + live walkthrough)	**Modal open (default):** aria-hides main+nav → 4/6 pages expose ZERO landmarks; only the modal's h2 reaches the SR (no page h1; skip link points at hidden main). **Modal removed (DOM-only):** every page exposes **4 landmarks** (nav/main/2× complementary) + a proper **h1** (view_document = 19 headings). **⇒ structure is sound; the auto-modal occludes it.** Positives: **0 unnamed interactives**, app-wide **Skip-to-main-content** link, proper ARIA **tree** doc list ✅ .
Page title (WCAG 2.4.2)	Descriptive on main_docs/issues/team_dir ✅ ; **view_document & my_week render `Ship | Ship`** (non-descriptive).

README "Section 508 / WCAG 2.1 AA compliant" claim → NOT SUPPORTED — on the **15 color-contrast AA failures + critical aria-required-children** (both independent of the modal). *Correction: an earlier draft called the modal an "inescapable keyboard trap / core 508"; the Q3-C dismissal experiment + live walkthrough showed it is escapable — severity reframed below.* Manual VoiceOver (Q3-B) attempted & blocked; its reproducible part is automated, qualitative flow flagged as Phase-2.

1. **High** — auto-opening modal occludes every authed page from keyboard/SR users until `Escape` (escapable, but hits every navigation for any user who owes a standup; one localized defect).
2. **Positive** — underlying a11y structure is good: 4 landmarks + proper h1 per page (modal-removed), 0 unnamed interactives, app-wide skip link, ARIA tree doc list.
3. **High** — README 508/WCAG-AA claim not supported (15 contrast AA + critical aria-required-children, independent of the modal).
4. **Low** — editor unlabeled (no role/aria-label); 2/6 pages have non-descriptive `Ship | Ship` titles (WCAG 2.4.2).

▸ **Improve** — PRD target: +10 Lighthouse on lowest page *or* fix **all Critical/Serious** on the 3 most important pages. (Lowest = view_document **91**; +10 = 101 > max 100, so the "fix all C/S" path is the realistic one.)

- **A H** stop the modal auto-opening (or render as a non-focus-stealing `role="status"` aside; if modal, return focus on close + don't reopen per-navigation) — fixes the occlusion; `cat7-sr-tree` modal-open vs removed diff re-proves it.
- **B M** fix all Critical/Serious on main_docs · view_document · my_week: `aria-required-children`, `listitem` structure, 15 `color-contrast` nodes (proven by `cat7-a11y` + `cat7-lighthouse`).
- **C L** contrast-only quick win: raise `.text-muted/*` token to clear 4.5:1 (removes 15 serious, +Lighthouse on my_week).
- **D M** add LHCI to CI so the Lighthouse metric stays measurable over time.

Recommended: A + B (A is the highest-leverage single fix — it un-occludes the sound structure underneath; B is the realistic PRD path, measurable via all three Cat-7 harnesses). Q3-C SR-tree raw → `docs/audit/raw/cat7-sr-before.txt`.

Audit recommendation — top 3 highest-value opportunities (the per-category "▸ Improve" options above are the full set; these are the strongest, each maps to a PRD ≥20% target, is independently reproducible & low-blast-radius): **(1)** trim list-response payloads + paginate `/api/documents` & `/api/issues` → P95 drop on ≥2 endpoints (`cat3-api.mjs`). **(2)** throttle the per-request `last_activity` write → fewer queries on ≥1 flow (`cat4-db.mjs`). **(3)** lazy-load editor-only deps → smaller initial bundle (`cat2-bundle.mjs`).

> **Supplementary deep static-review findings** (full read-through beyond the 7 harnesses — diagnoses by code inspection, each `file:line`'d & verified; see `AUDIT_REPORT.md` §Supplementary). Headline: **S1 [High]** migration 033 fails on a fresh DB (clean clone can't provision) · **S2 [High]** soft-delete leaks — backlinks/dashboard/week-metrics count trashed docs · **S3 [High]** hot JSONB filters unindexed (10× latent) · **S4 [High]** SIGTERM drops in-flight Yjs saves (data-loss every deploy) · **S12 [Med-High]** persisted query cache not identity-scoped (cross-user exposure on shared browser). Cross-cutting ops: no CI, non-hermetic/root Docker, committed deploy bundles (no secrets — verified). Positives: strong Terraform/WAF, consistent multi-tenant isolation, parameterized SQL, no secrets in history.

Phase-1 gate: **7 / 7 categories baselined**; each table mirrors the PRD's prescribed metrics, and each category lists improvement **options** mapped to its PRD Improvement Target. These are audit *recommendations* (Phase-1 diagnostic output) — **no application code was changed**; only reproducible measurement instruments + deterministic test data. Full methodology, raw evidence, and the cross-category ranked findings live in `AUDIT_REPORT.md`. Each recommended fix would be executed and proven before/after with its named `scripts/audit/catN-*` harness under the fixed condition of record.